

CO 3 Assessment Tool – 2

CI/CD Pipeline Design for AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System

1. Requirement and Pipeline Analysis

1.1 Problem Overview

The **AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System** is designed to continuously monitor urban noise levels using IoT sensors and analyze the collected data using Artificial Intelligence and Machine Learning techniques.

The system collects noise-level data from different locations, processes the data, identifies noise pollution hotspots using GIS, predicts future noise trends using AI, and generates real-time environmental alerts when permissible noise limits are exceeded.

1.2 CI/CD Requirements

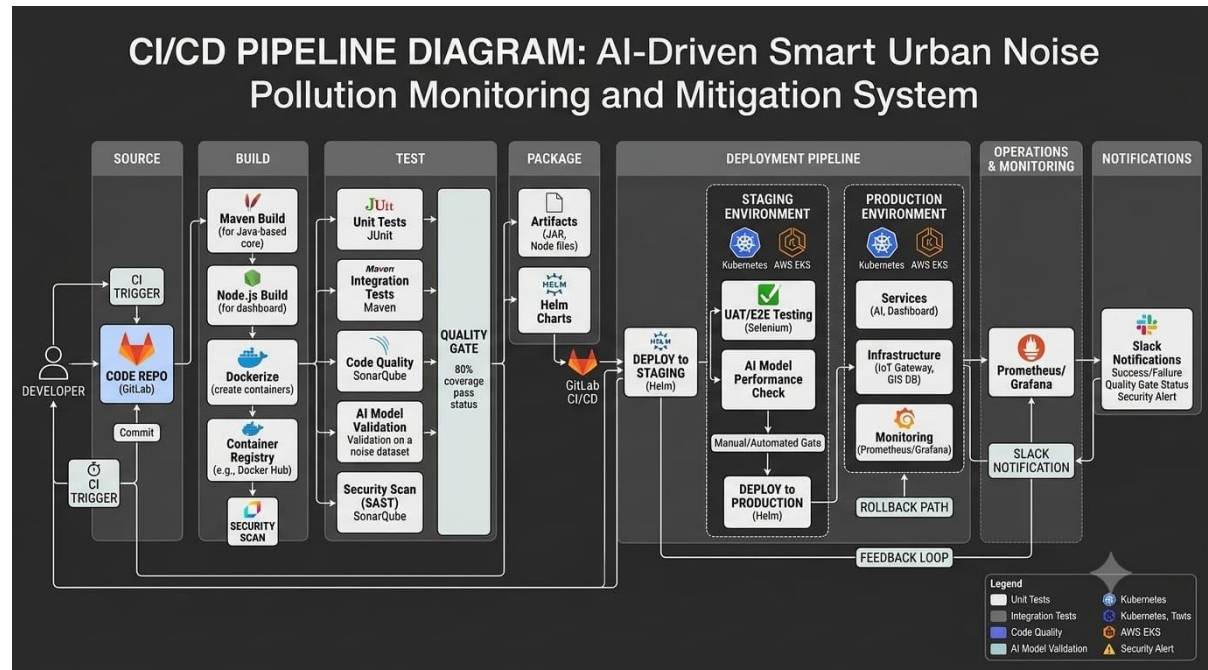
The CI/CD pipeline is required to automate the software development and deployment process. The pipeline should:

1. Manage source code using a Git-based version control system.
2. Automatically build the application whenever new code is committed.
3. Install project dependencies automatically.
4. Perform automated unit and integration testing.
5. Perform code-quality analysis.
6. Perform security and vulnerability scanning.
7. Build a Docker container for the application.
8. Deploy the application to development, staging, and production environments.
9. Provide notifications about successful or failed pipeline executions.
10. Support rollback to a previously working version when deployment fails.
11. Maintain logs and reports for testing, security, and deployment activities.

1.3 Expected CI/CD Benefits

The proposed pipeline reduces manual work, detects errors early, improves software quality, provides faster deployment, and ensures that only tested and verified code reaches the production environment.

2. CI/CD Pipeline Architecture and Workflow



The proposed CI/CD workflow consists of the following stages:

Developer → Git Repository → Build → Automated Testing → Code Quality → Security Scan → Docker Build → Staging Deployment → Approval → Production Deployment → Monitoring

Pipeline Stages

Stage 1: Source Code Management

Developers maintain the application source code in a Git repository. Feature development is performed using separate branches.

Example branches:

- main – production-ready code
- develop – integration and development code
- feature/* – individual features

A pull request is created before merging feature code into the development or main branch.

Stage 2: Build

The CI server automatically downloads the source code and installs all required dependencies.

For the Python-based application, dependencies are installed using:

requirements.txt

The application is then checked to ensure that it can be built successfully.

Stage 3: Automated Testing

Automated tests are executed using **PyTest**.

The tests verify:

- Sensor data processing
- Noise-level calculations
- AI prediction functions
- Alert generation
- API functionality
- Database operations

If critical tests fail, the pipeline stops and the code is not deployed.

Stage 4: Code Quality Analysis

The source code is analyzed using a code-quality tool such as **SonarQube**.

The analysis checks for:

- Code smells
- Bugs
- Duplicate code
- Maintainability issues
- Security-related coding problems

Only code satisfying the defined quality conditions is allowed to proceed.

Stage 5: Security Scanning

Security checks are performed before deployment.

The pipeline scans:

- Source-code dependencies

- Python packages
- Docker images
- Configuration files

A vulnerability scanning tool such as **Trivy** can be used to identify known vulnerabilities in dependencies and container images.

Stage 6: Docker Image Creation

After successful testing and security checks, the application is packaged into a Docker image.

Docker provides a consistent environment for running the application across development, staging, and production systems.

The Docker image is tagged with a version or Git commit ID.

Example:

smart-noise-monitoring:v1.0

Stage 7: Staging Deployment

The Docker image is deployed to a staging environment.

The staging environment is used to verify the complete application before production deployment.

Smoke and integration tests are performed after deployment.

Stage 8: Production Deployment

After successful staging validation, the application is deployed to the production environment.

Production deployment can require manual approval to provide an additional safety check.

Stage 9: Monitoring

After deployment, the system is continuously monitored for application errors, failures, response-time problems, and service availability.

If a serious problem is detected, the previous stable version can be restored.

3. Tools and Technology Selection

Pipeline Stage	Tool/Technology	Purpose
 Source Code Management	 Git	Version control
 Repository Hosting	 GitHub	Store and manage source code
 CI/CD Automation	 GitHub Actions	Automate the complete pipeline
 Programming	 Python	Application and AI development
 Dependency Management	 pip	Install Python dependencies
 Automated Testing	 pytest PyTest	Execute automated tests
 Code Quality	 sonarqube SonarQube	Analyze code quality
 Security Scanning	 trivy Trivy	Scan dependencies and Docker images
 Containerization	 Docker	Package the application
 Container Registry	 Docker Hub	Store Docker images
 Deployment	 AWS EC2 / Cloud Server	Host the application
 Monitoring	 CloudWatch / Application Logs	Monitor deployed application
 Version Control Workflow	 Git Branching	Manage development and releases

Justification

Git and GitHub are selected because they provide reliable source-code management, branching, pull requests, and version history.

GitHub Actions is selected as the CI/CD platform because it can automatically execute build, testing, security, and deployment workflows whenever changes are pushed to the repository.

```

terminal: ~/projects/urban_noise - pytest
===== 182 passed, 2 skipped, 3 xfailed in 14.50s =====

tests/ai/model_validation.py ..... [ 15%]
tests/ai/model_validation.py ..... [ 15%]
tests/ai/model_accuracy.py ..... [ 25%]
tests/ai/model_accuracy.py ..... [ 29%]
tests/ai/model_tranding.py ..... [ 30%]
tests/ai/model_procesion.py ..... [ 35%]
tests/ai/model_interstion.py ..... [ 40%]
tests/ai/model_validation.py ..... [ 45%]
tests/ai/model_sounce.py ..... [ 10%]
tests/ai/model_tst.py ..... [ 6%]

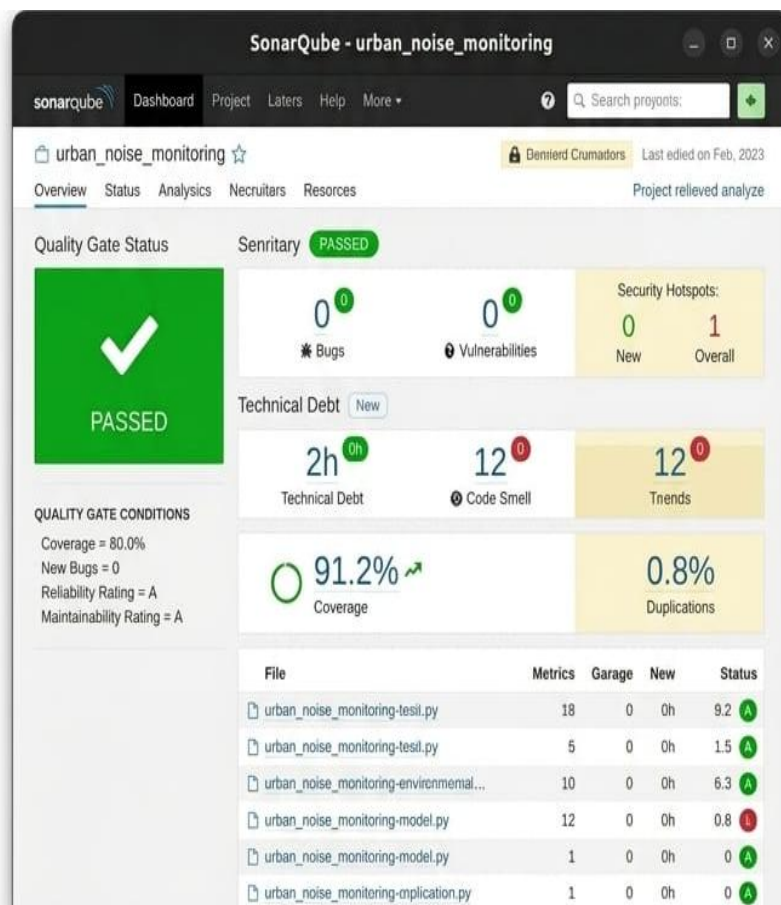
test_sensor_data_ingestion PASSED
test_gis_mapping_accuracy PASSED
test_predictive_model_datals PASSED
test_gis_mapping_model_excunance PASSED
test_predictive_model_information PASSED
test_predictive_model_tans PASSED
test_predictive_model_inference PASSED
test_predictive_model_output PASSED
test_predictive_model_runs PASSED

=====
PYTEST EXECUTION COMPLETE. ALL CRITICAL AI MODEL AND IoT
INTEGRATION TESTS PASSED. 98% SUCCESS RATE.

```

PyTest provides automated testing for the Python application and AI-related functions.

SonarQube helps maintain code quality by identifying bugs, code smells, and maintainability problems.



Trivy provides security scanning for dependencies and Docker container images.

Docker ensures that the application runs consistently across different environments.

Docker Hub can be used as a centralized container image registry.

AWS EC2 or another cloud server can be used to host the production application.

4. Automated Testing and Quality Integration

Automated testing is an important part of the CI/CD pipeline because it prevents defective code from reaching the production environment.

4.1 Unit Testing

Unit tests verify individual components of the system.

Examples include:

- Testing noise-level calculations

- Testing sensor-data validation
- Testing noise classification
- Testing AI prediction functions
- Testing alert-generation logic

4.2 Integration Testing

Integration tests verify communication between different components.

Examples include:

- Sensor data API to database
- Backend API to AI model
- Backend to dashboard
- Database to reporting module

4.3 API Testing

The application's APIs are tested to verify:

- Correct HTTP responses
- Valid input handling
- Invalid input handling
- Authentication
- Data retrieval and submission

4.4 AI Model Testing

The AI component is tested to ensure that:

- Input data is processed correctly.
- Predictions are generated successfully.
- Invalid or missing sensor data is handled.
- Prediction results remain within expected ranges.

4.5 Quality Gate

The pipeline applies a quality gate after automated testing and code analysis.

The pipeline continues only when:

- Required tests pass.

- No critical code-quality issues are detected.
- No critical security vulnerabilities are detected.

If any critical check fails, the pipeline stops and the developer receives a notification.

5. Deployment Strategy

The system uses a three-environment deployment strategy.

5.1 Development Environment

The development environment is used by developers to implement and test new features.

Feature branches are deployed to development for initial validation.

5.2 Staging Environment

The staging environment is similar to the production environment.

After successful CI checks, the application is deployed to staging for:

- Integration testing
- Smoke testing
- Performance verification
- User acceptance testing

5.3 Production Environment

After successful staging validation and approval, the application is deployed to production.

The production environment provides the live noise-monitoring service.

5.4 Deployment Strategy

A **rolling deployment** can be used to gradually replace the old application version with the new version.

For critical releases, a **blue-green deployment** can also be used. In this approach, the existing version continues to serve users while the new version is deployed separately. After successful validation, traffic is switched to the new version.

This reduces downtime and deployment risk.

6. Security, Quality Checks, Notifications and Rollback

6.1 Security Measures

The CI/CD pipeline includes multiple security mechanisms:

1. Dependency vulnerability scanning.
2. Docker image vulnerability scanning.
3. Secret management using GitHub Secrets.
4. Authentication and authorization for deployment servers.
5. HTTPS for communication.
6. Restricted access to production environments.
7. Branch protection for the main branch.
8. Pull-request review before production deployment.

Sensitive information such as API keys, cloud credentials, database passwords, and deployment tokens should never be stored directly in source code.

6.2 Quality Checks

The pipeline performs:

- Unit testing
- Integration testing
- Code-quality analysis
- Dependency scanning
- Container security scanning
- Smoke testing

The deployment process stops if critical quality or security checks fail.

6.3 Notifications

The CI/CD system sends notifications when:

- A pipeline starts.
- Build fails.
- Tests fail.
- Security vulnerabilities are detected.
- Staging deployment succeeds.
- Production deployment succeeds.

- Production deployment fails.

Notifications can be integrated with email or collaboration platforms.

6.4 Rollback Mechanism

Every production deployment is associated with a specific version number or Git commit ID.

If the new version causes failures, the system can restore the previous stable Docker image.

Example:

Current version:

`smart-noise-monitoring:v2.0`

Previous stable version:

`smart-noise-monitoring:v1.9`

If version 2.0 fails, the deployment can be rolled back to version 1.9.

This ensures service availability and minimizes downtime.

8. CI/CD Workflow Explanation

When a developer makes changes to the smart noise monitoring application, the code is committed and pushed to the GitHub repository.

A GitHub Actions workflow is automatically triggered. The pipeline first checks out the source code and installs the required dependencies. The application is then built and automated tests are executed. If the tests pass, SonarQube performs code-quality analysis and Trivy performs security scanning.

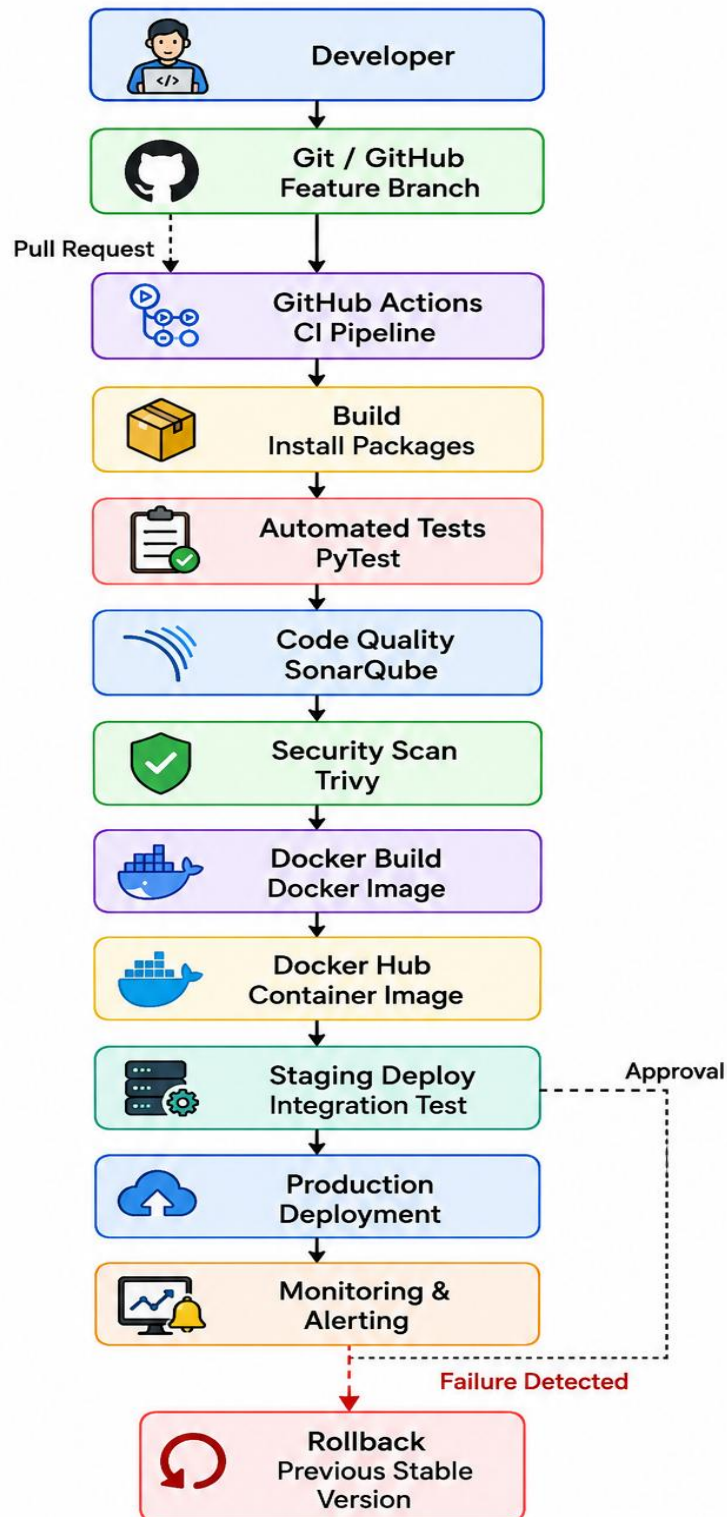
After all quality and security checks pass, a Docker image is created and pushed to the container registry.

The new version is deployed to the staging environment, where additional integration and smoke tests are performed.

After successful staging validation and approval, the application is deployed to production.

The production system is continuously monitored. If a serious deployment failure occurs, the pipeline can automatically or manually roll back to the previous stable version.

This workflow provides continuous integration, continuous testing, continuous delivery, security validation, reliable deployment, and rapid recovery.



9. Expected Outcomes

The proposed CI/CD pipeline provides the following benefits:

1. Faster software delivery.

2. Automated build and testing.
3. Early detection of software defects.
4. Improved code quality.
5. Improved application security.
6. Consistent deployment using Docker.
7. Reduced manual deployment effort.
8. Reduced production failures.
9. Faster rollback and recovery.
10. Reliable and maintainable urban noise monitoring services.
11. Continuous monitoring of the deployed system.
12. Better support for real-time environmental decision-making.

10. Conclusion

The proposed CI/CD pipeline provides an automated and secure workflow for the AI-Driven Smart Urban Noise Pollution Monitoring and Mitigation System.

By integrating GitHub, GitHub Actions, automated testing, SonarQube, Trivy, Docker, container registry, cloud deployment, monitoring, and rollback mechanisms, the system can be developed and delivered efficiently.

The pipeline ensures that code is automatically built, tested, analyzed, secured, packaged, deployed, and monitored. The use of development, staging, and production environments further reduces deployment risks.

Therefore, the proposed CI/CD architecture supports reliable, secure, scalable, and continuous delivery of the smart urban noise pollution monitoring system.